目录

后记：回到 Agent = LLM + 上下文 + 工具 **1**

两朵乌云 1

模型与 Agent 的共同演进 2

后记：回到 Agent = LLM + 上下文 + 工具

本书开篇提出了一个公式：**Agent = LLM + 上下文 + 工具。**全书十章，都在这三个词里展开。

第一章建立公式的三层理解——实现层、直觉层、学术层，并给出从工作流到自主 Agent 的编排光谱。随后的章节沿“构建—评估与进化—协作”逐步展开。

- **构建 Agent（第二至六章）。**上下文工程决定 Agent 在一次任务中看到什么，记忆与知识库把信息扩展到多次会话；工具定义它能做什么，代码生成则提供创造新工具与新系统的元能力；交互一章再把观察与动作空间从文本回合制推向语音、GUI 与物理世界。
- **评估与进化（第七至九章）。**评估把表现变成可信信号，后训练把高维能力写入模型参数，持续进化再把生产经验转化为知识、指令、程序或参数的受控更新。
- **协作（第十章）。**多 Agent 协作进一步改变上下文、工具与责任的组织方式。

这三个层次并不是相互独立的书架。第九章尤其依赖前文的全部基础：没有轨迹和知识系统，经验无处保存；没有代码能力，Agent 无法修改工具与 Harness；没有评估，系统更无法判断一次修改究竟是进步还是退化。它由此成为全书从“怎样构建 Agent”转向“怎样让 Agent 长期变好”的汇合点。

两朵乌云

1900 年，开尔文说物理学晴朗的天空上还飘着两朵乌云——后来，一朵变成了相对论，另一朵变成了量子力学。今天 Agent 的天空同样称不上晴朗，我也看到两朵乌云。

第一朵乌云，是 Agent 如何流式地、实时地与环境交互。今天绝大多数 Agent 仍是按轮次（turn-by-turn）的“请求—应答”模式：你说完一句，它想一整段，再一次性吐出结果。但真实世界不会停下来等它想完——话会被打断，画面在持续变化，邮件在不断到达。一个真正“活着”的 Agent，应该能边听边想、边说边想，能在你话说到一半时就开始规划，也能在没人吩咐时主动发现“这封邮件该处理了”。走向这种实时性有两条路，往往并行推进：一是**架构上做快慢分离**——实时与智能几乎是两条正交的轴，单一模型难以兼顾，于是让前台快模型维持对话节奏、后台慢模型负责深度思考；二是**把推理本身做快**——当 decode 速度足够高，按轮次的等待就短到近乎消失，turn-by-turn 与“实时”的界限也随之模糊。这条路正被芯片与推理引擎快速推进：小米 MiMo 已让一个 1T 参数模型在单个 8 卡节点上把生成速度推过 1000 token/s[1]，而把整个模型直接固化进芯片的专用方案（如 Taalas HC1）更是把 80 亿参数模型推到约 17000 token/s、响应低于 100 毫秒[2]。当模型每秒能吐出上千个字，“想完再说”和“边想边说”的体验差距就被抹平了。

第二朵乌云，是 Agent 如何像人一样，从与环境交互的成功与失败中持续积累经验。今天的模型更像一个记性极好、却学不会新东西的天才：训练时把人类知识背得滚瓜烂熟，上岗后却几乎不再成长——每次任务结束，那些踩过的坑、试出来的窍门，大多随着上下文一起被丢掉。这究竟是不是一个真问题，取决于两种针锋相对的假设。

一种是 **“小世界假设”**：一个足够大的模型——比如几万亿参数——本就装得下物理世界里几乎所有重要的通用知识，学一次就够。持这种看法的人（不乏 OpenAI、Anthropic 的研究者）会指出，AI 今天唯独在编程上最强，并不是因为代码对模型有什么特殊，而是因为编程是人类最开放的领域：海量开源代码摆在那里，可供学习；而绝大多数行业压根没有公开的信息与数据。于是前沿实验室真正在做的，是一家家去与各行各业合作，把各自的专业能力“蒸馏”进同一个大模型。按这种观点，瓶颈既不在模型的容量、也不在它学不学得会，而在数据够不够。把数据喂进去、训练一次，问题就解决了。

[1] 小米 MiMo-V2.5-Pro-UltraSpeed 通过 FP4 量化、DFlash 并行推测解码与 TileRT 推理系统的模型—系统协同设计，在单个通用 8-GPU 节点上首次把 1T 参数模型的生成速度推过 1000 token/s。见小米 MiMo 官方技术博客 “Pushing 1T-Parameter Model Generation Speed to 1000 TPS” , 2026. https://mimo.xiaomi.com/blog/mimo-tilert-1000tps

[2] Taalas HC1 将 Llama 3.1 8B 整个模型固化进 6nm 芯片，实现约 17000 token/s、响应低于 100 毫秒；代价是芯片只能运行被固化的那个模型，模型更新需重新流片。见 Karl Freund, “Taalas Launches Hardcore Chip With ‘Insane‘ AI Inference Performance,” Forbes, 2026. https://www.forbes.com/sites/karlfreund/2026/02/19/taalas-launches-hardcore-chip-with-insane-ai-inference-performance/。

但 **"大世界假设"** 指向了单靠"训练一次"补不上的一层：属于某个具体用户、某家具体公司的知识。特定公司的代码规范、做 PPT 的口味、某个客户特有的脾气不在任何训练语料里，而且时时在变；要贴合这个由无数具体情境拼成的"大世界"，模型只能在上岗之后持续学习，没法指望出厂时一次配齐。这正是第三章的记忆与第九章的持续进化在摸索的方向：把经验写成知识文档、指令或程序，还是经过筛选后用于更新模型参数？更进一步，"RSI"（递归自我进化）和"AI for Science"都在推动 Agent 走到没有现成答案的前沿；在那里，它只能从一次次实验的成败中自主学习，而不是事事回头问人。所以，模型最强的能力，终将不是记住，而是学习与适应。

这两朵乌云，都不是靠某一次模型升级就能凭空吹散的。要理解它们最终会怎样被跨越，得先看清一件事：模型和 Agent，从来不是上下游，而是一起往前走的。

模型与 Agent 的共同演进

回头看那些 harness 里层层叠叠的兜底逻辑——多级上下文压缩、失败数千次才熔断的重试、悲观地默认"不安全"的权限判断——每一段看似丑陋的"屎山"，记录的都是模型此刻还做不稳的地方。当下一代模型把这些约束内化，对应的代码就可以删掉；而模型之所以能内化，又正是因为 Agent 早已在真实业务里替它把这些坑趟了一遍，沉淀成了下一轮训练的信号。用户提出真实的难题，应用层用 harness 把模型暂时做不好的事补上，这些补救再反过来变成模型下一次迭代的训练信号。这是一个自我强化的飞轮。

这个飞轮，也回答了第一章悬下的那个问题：**模型会不会最终吃掉 Harness?** 本书的观点是：**会，但不是一次性吃掉，而是一层一层地吃，并且没有吃完的那一天。**模型每稳定内化一种能力，对应的 Harness 层就可以删掉——第六章的交互模型就是这样一个例子：打断、插话这些曾经要靠外挂 harness 才能拼出的行为，如今被直接做进了模型内部。但这个"吃"永远不会完结。一是训练以月计，模型等得起，业务等不起；二是模型无法内化真实业务中所有的约束与偏好，总有一层最新的边界需要外部逻辑兜底；三是每一代模型都会打开新的能力前沿，而前沿处恰恰是模型最做不稳的地方。所以 Harness 不会消失，它只是随着模型，不断向新的前沿迁移。这也正是《苦涩的教训》在 Agent 时代的读法：通用方法终将胜出，但"终将"二字里的每一段路，都是 Harness 铺出来的。

而飞轮转得最快的地方，是同时握住两端的人。Anthropic 用 Claude Code 做的，正是让自家模型和自家 harness 互相喂养、共同进化：模型知道 harness 会怎样调用它，harness 也清楚模型的边界在哪，两端的每一次改动都能立刻反馈给对方。曾有人做过一个实验，不换模型、只改 harness，任务准确率就从 52.8% 跳到 66.5%。这既说明 harness 今天的杠杆有多大，也提醒你：它之所以有这么大杠杆，恰恰是因为模型还没走到那一步。也正因如此，这条飞轮本身，就是这个时代最深的一条护城河：真实业务、反馈数据与模型迭代咬合得越紧，别人越难从外部追上。

这对你意味着什么，取决于你站在飞轮的哪一端。如果你在造模型，护城河就是把这条飞轮转起来——让真实场景的反馈尽快回流到训练里。如果你在模型之上造应用，harness 是你短期最锋利的技术杠杆，但要清醒：模型每内化一层约束，就会顺手抹平一批只靠 harness 建立的优势。应用层真正长久的护城河，往往在技术之外——独占的数据、稳固的渠道、用户的信任、网络效应，以及必须由人与 Agent 协作的物理世界场景等。把 harness 用来争取时间，把这段时间用来构筑技术之外的壁垒，才是稳妥的打法。

所以，不必焦虑手里的框架会不会过时。模型每几个月迭代一次，具体的 API、产品和榜单都会翻篇，但"看到什么、能做什么、如何验证做得对不对"这三个问题不会过时——它们描述的不是某个模型的用法，而是一个智能系统与世界交互的基本方式。掌握了它们，无论下一代模型带来什么新能力，你都知道该把它放进公式的哪个位置，也能一眼看出，它离吹散那两朵乌云还有多远。

Agent 技术仍在飞速演进，一本书追不上所有变化。但如果这本书让你带走的不是某个 API 的具体用法，而是一套能在技术浪潮里保持清醒的判断力，那它就完成了使命。本书全部正文、配图与配套实验代码都是开源的，欢迎你去仓库里把实验亲手跑一遍、提 issue 和 PR。而 Agent 最迷人的地方，正在于它能通过写代码创造新的能力，甚至改进自己；读到这里，你已经握住了"创造"的原则。接下来，去造点什么吧。